



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

Prompt Engineering for C and C++ 2026-2027

Course: B.Tech CSE (Core)

Registration number: 25BCON1594

Section: I

Semester: III

Submitted to: Mr. Ajay Sharma

Submitted by: Chehak Gupta

Session 08 Homework: Red Before Green

1. Specification (Custom Python Program)

Requirement: Write a 6+ row specification for your own Python program.

Program: E-commerce Discount Calculator

1. `cart_total < 0` must raise a `ValueError` for invalid amounts.
2. `cart_total` between 0 and 50 (inclusive) must return a 0.0(0%) discount.
3. `cart_total` between 51 and 100 (inclusive) must return a 0.10 (10%) discount.
4. `cart_total > 100` must return a 0.20 (20%) discount.
5. If `is_vip=True`, the final calculated discount percentage must have an additional flat 0.05 (5%) added to it.
6. The maximum possible combined discount cannot exceed 0.25 (25%).

2. Seeded Bug Classes & The Red-Green Cycle

Requirement: Seed three bug classes into a copy, write spec-based tests, prove red, then fix and prove green.

- **Boundary Bug:** Used `>= 50` for the 10% discount instead of `> 50`.
- **Unreachable Branch:** Placed `> 100` check below the `> 50` check, so \$120 orders only got 10%.
- **Arithmetic Bug:** VIP discount added 0.50 (50%) instead of 0.05 (5%).

Original Code:

```
student_utils.py  test_student_utils.py  discount.py  test_discount.py X
HW > test_discount.py > ...
1  import unittest
2  from discount import calculate_discount
3
4  class TestDiscount(unittest.TestCase):
5      def test_negative_total_raises_error(self):
6          with self.assertRaises(ValueError):
7              calculate_discount(-10)
8
9      def test_exactly_fifty_gets_zero_discount(self):
10         self.assertEqual(calculate_discount(50), 0.0)
11
12     def test_seventy_gets_ten_percent(self):
13         self.assertEqual(calculate_discount(70), 0.10)
14
15     def test_one_hundred_gets_twenty_percent(self):
16         self.assertEqual(calculate_discount(100), 0.20)
17
18     def test_vip_gets_five_percent_bonus(self):
19         self.assertEqual(calculate_discount(20, is_vip=True), 0.05)
20
21     def test_discount_capped_at_twenty_five(self):
22         self.assertEqual(calculate_discount(150, is_vip=True), 0.25)
23
24 if __name__ == '__main__':
25     unittest.main()
```

HW RED STAGE-Failing Tests

```
PS C:\Users\DESKTOP\OneDrive\Documents\session08> & C:/Users/DESKTOP/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DESKTOP/OneDrive/Documents/session08/HW/test_discount.py
Traceback (most recent call last):
  File "c:/Users/DESKTOP/OneDrive/Documents/session08/HW/test_discount.py", line 2, in <module>
    from discount import calculate_discount
ImportError: cannot import name 'calculate_discount' from 'discount' (c:/Users/DESKTOP/OneDrive/Documents/session08/HW/discount.py)
PS C:\Users\DESKTOP\OneDrive\Documents\session08> []
```

HW GREEN STAGE - Passing Tests

```
PS C:\Users\DESKTOP\OneDrive\Documents\session08> & C:/Users/DESKTOP/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DESKTOP/OneDrive/Documents/session08/HW/test_discou
.....
Ran 6 tests in 0.000s

OK
PS C:\Users\DESKTOP\OneDrive\Documents\session08>
```

3. Implementation-Based Test Experiment & Conclusion

Requirement: Generate tests from broken code, compare both suites, and report the actual experiment result.

AI Prompt Used:

"Read this broken discount. pyfile. Ignore all specifications. Write a complete test suite based strictly on what this code currently does."

Experiment Result:

When I ran the AI-generated tests against the broken code, **they passed completely (Green Stage) without me fixing anything.**

Conclusion (Rejecting False Confidence): This experiment proves the phenomenon of **"Misleading Green"**. Because the AI used the wrong source (reading what

the code currently does instead of what it should do), the generated tests explicitly asserted the incorrect behavior. For example, the AI wrote a test asserting that a \$50 order *should* get a 10% discount due to my seeded boundary bug.

Fixing is not proving. The statement "The AI fixed it" is dangerous because the AI changed something I had not independently verified. Evidence is stronger than confidence; a test that has never failed has proved absolutely nothing. We must write tests from the specification, independently of the code.

4. Code Implementation

Fixed discount. py:

Python

```
def calculate_discount(cart_total, is_vip=False):
    if cart_total < 0:
        raise ValueError("Invalid total")

    if cart_total > 100: # Fixed: Unreachable branch moved up
        base_discount = 0.20
    elif cart_total > 50: # Fixed: Boundary bug (was >= 50)
        base_discount = 0.10
    else:
        base_discount = 0.0

    if is_vip:
        base_discount += 0.05 # Fixed: Arithmetic bug (was 0.50)

    return min(base_discount, 0.25)
```

test_discount.py:

```
Python
import unittest
from discount import calculate_discount

class TestDiscount(unittest.TestCase):
    def test_negative_total_raises_error(self):
        with self.assertRaises(ValueError):
            calculate_discount(-10)

    def test_exactly_fifty_gets_zero_discount(self):
        self.assertEqual(calculate_discount(50), 0.0)

    def test_seventy_gets_ten_percent(self):
        self.assertEqual(calculate_discount(70), 0.10)

    def test_one_twenty_gets_twenty_percent(self):
        self.assertEqual(calculate_discount(120), 0.20)

    def test_vip_gets_five_percent_bonus(self):
        self.assertEqual(calculate_discount(20, is_vip=True), 0.05)

    def test_discount_capped_at_twenty_five(self):
        self.assertEqual(calculate_discount(150, is_vip=True), 0.25)

if __name__ == '__main__':
    unittest.main()
```